Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Аналітика великих даних»

Практичне заняття №2

«Розподілена обробка великих даних за допомогою Apache Spark»

Виконав:
ст. гр. ДМм-26-1
Зелінський В.С.

Прийняв:
Золотухін О.В.

Харків – 2026

Розподілена обробка великих даних за допомогою Apache Spark

Мета роботи

Ознайомитися з архітектурою та принципами роботи Apache Spark як платформи розподіленої обробки великих даних; отримати практичні навички завантаження, очищення, трансформації та агрегації великого набору даних за допомогою Spark DataFrame і PySpark на прикладі хвилинних свічок BTC/USD (Bitcoin Historical Data, 7 816 537 записів за 2012–2026 роки); навчитися виконувати аналітичні операції над цими даними та використовувати Spark SQL для формування аналітичних запитів, а також порівняти Spark з Hadoop MapReduce за моделлю обчислень, використанням пам'яті, продуктивністю, зручністю програмування та можливостями повторного використання даних.

Короткі теоретичні відомості

У роботі використано Spark Core і Spark SQL. Програма PySpark починається зі `SparkSession`; у режимі `local[*]` драйвер і виконавці працюють в одній JVM, а паралелізм дають потоки за кількістю ядер. DataFrame – розподілена таблиця зі схемою, поділена на розділи; кожен розділ обробляє одна задача. За схемою оптимізатор Catalyst будує фізичний план і відкидає непотрібні стовпці, тому DataFrame API і Spark SQL дають однаковий план.

Трансформації (`filter`, `withColumn`, `groupBy`) лініві: вони лише доповнюють план, а обчислення запускає дія (`count`, `collect`, запис). План ділиться на етапи по межах обміну (shuffle): вузькі операції виконуються конвеєром, а групування, сортування і вікна з `partitionBy` потребують обміну. `persist()` зберігає обчислені розділи в пам'яті для повторних запитів. Hadoop MapReduce, з яким порівнюється Spark, записує

проміжні результати кожного завдання на диск, і наступне завдання знову читає дані з диска.

Результати виконання практичного заняття

1 Підготовка середовища

Машина: Intel Xeon 2,10 ГГц (4 логічні ядра), 15 ГБ пам'яті, Ubuntu 24.04; встановлено OpenJDK 21.0.12.1, Python 3.13.18, Apache Spark і PySpark 4.3.0, pandas 3.0.5, Jupyter Notebook 7.6.5. Перевірки версій і пробне групування пройшли без помилок (рисунок 1).

```
bash — pz2
$ python code/pz2_spark.py
# фрагмент виводу: версії програмного забезпечення та конфігурація SparkSession
Java                                        openjdk version "21.0.12.1" 2026-08-18
Spark                                       4.3.0
PySpark                                     4.3.0
Python                                      3.13.18
pandas                                      3.0.5
numpy                                       2.5.3
matplotlib                                  3.11.2
psutil                                      7.2.2
playwright                                  1.65.0
Chromium                                    Google Chrome for Testing 153.0.8010.12
OS                                          Linux-6.18.44-fc-v378-x86_64-with-glibc2.39
CPU                                         Intel(R) Xeon(R) Processor @ 2.10GHz
CPU_logical_cores                           4
RAM_GB                                      15
spark.master                                local[*]
spark.defaultParallelism                    4
spark.driver.memory                         8g
spark.sql.shuffle.partitions                16
spark.sql.adaptive.coalescePartitions.enabled false
spark.sql.session.timeZone                  UTC
spark.ui.url                                http://192.0.2.2:4040
seed                                        42
run_started                                 2026-11-11 23:25:01
loadavg_at_start                            8.60 7.66 6.37 5/948 21482
```

Рисунок 1 – Версії компонентів і параметри SparkSession

`spark.defaultParallelism = 4` підтверджує використання всіх ядер. Розділів обміну задано 16 замість типових 200, бо на 4 ядрах дрібні задачі дали б більше накладних витрат, ніж роботи; злиття малих розділів вимкнено, щоб кеш зберігав 16 розділів; зона UTC узгоджує межі діб з П31.

2 Завантаження даних

Файл прочитано з `inferSchema=True` і з явною схемою `StructType` (`Timestamp` – `LongType`, решта – `DoubleType`); результат наведено на рисунку 2.

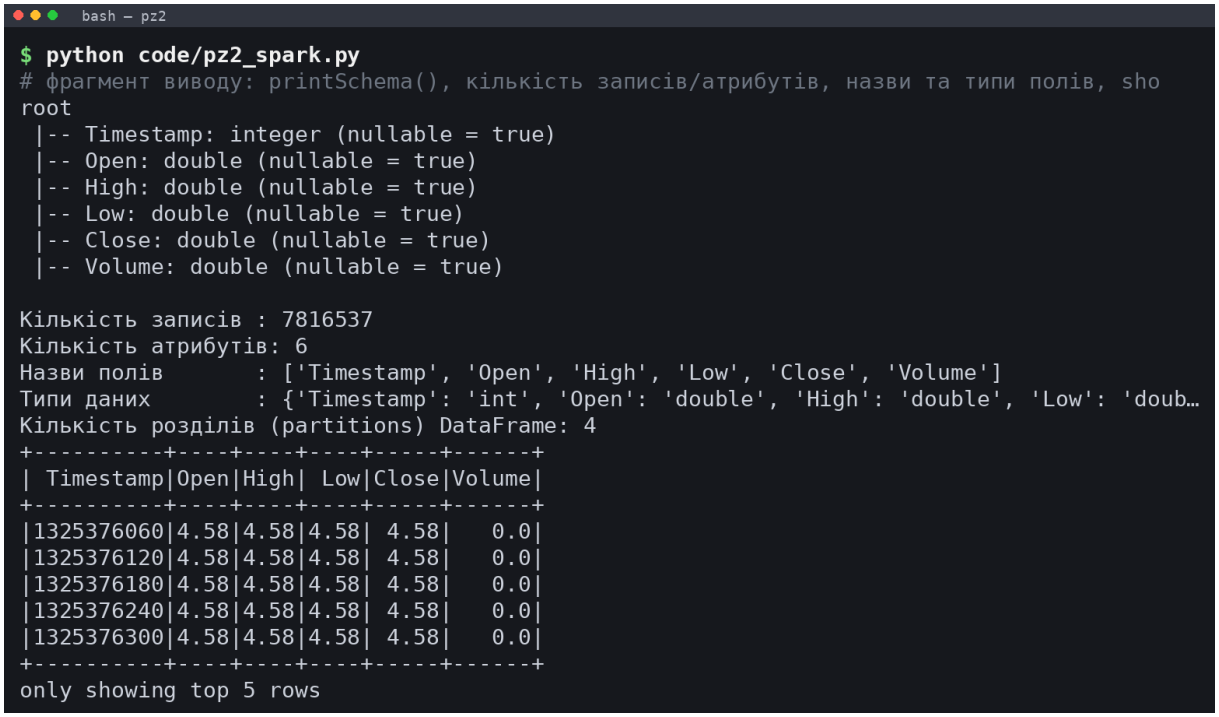


Рисунок 2 – Схема, кількість записів і атрибутів, перші рядки DataFrame

DataFrame містить 7 816 537 записів і 6 атрибутів (`Timestamp`, `Open`, `High`, `Low`, `Close`, `Volume`) у 4 розділах; перевірка `isNull` і `isnan` дала 0 пропусків. Для `Timestamp` виведення типів дало 32-бітний `int`, бо всі мітки поки менші за 2^{31}; явна схема задає `bigint`, що не залежить від діапазону. Дія, що розбирає всі шість стовпців, з `inferSchema` тривала 10,744 с (медіана, 9,903–16,929) проти 5,014 с (4,949–5,124) з явною схемою, тобто у 2,14 разу довше через додатковий прохід файлом. `count()` для такого порівняння непридатний: він не розбирає стовпців (1,056 с).

3 Попередня обробка

3.1 Пропуски, дублікати, типи та похідні атрибути

`dropna()` і `dropDuplicates(["Timestamp"])` залишили 7 816 537 рядків (0 дублікатів). Типи приведено явною схемою і `timestamp_seconds`, з якого в UTC виділено `datetime`, `date`, `year`, `month`, `hour`. Сформовано `is_flat` (нульовий обсяг), `range`, `rel_range = (High − Low) / Open`, логарифмічну дохідність `log_ret` і її 60-хвилинне стандартне відхилення `vol60`:

```
w = Window.partitionBy("year", "month").orderBy("Timestamp")
d = d.withColumn("log_ret", F.log("Close") - F.log(F.lag("Close").over(w)))
d = d.withColumn("vol60", F.stddev("log_ret").over(w.rowsBetween(-59, 0)))
```

Вікно секціоновано за (year, month), бо без `partitionBy` Spark зводить 7,8 млн рядків в один розділ і рахує вікна однією задачею. Ціна – крайовий ефект: у кожній із 179 груп перша хвилина має `log_ret = NULL`, а 10 740 рядків (0,14 %) мають у рамці `vol60` менше 60 точок, що для добових агрегатів несуттєво.

3.2 Хвилини з нульовим обсягом

Хвилин з `Volume = 0` – 1 312 419 (16,79 %); їх ціни дорівнюють закриттю попередньої хвилини, тобто це заповнення сітки, а не угоди. Частка flat-хвилин становить 94,95 % у 2012 році і не перевищує 3,95 % з 2018 року; 6–8 січня 2015 року угод не було. Видалення цих хвилин зруйнувало б `lag("Close")` і 60-хвилинну рамку, тому їх лише позначено `is_flat` і відфільтровано в моментах (`is_flat = 0`) і рейтингах діб (`flat_share < 0,5`). Поріг відсікає 2012 рік, зберігає 2013–2016 роки (24,24–39,16 %) і залишає 4876 з 5429 діб.

3.3 Ліниві обчислення, план виконання і кешування

Функція `build()` описує весь конвеєр (читання, `dropna`, `dropDuplicates`, десять `withColumn`, два вікна) без обчислень. Над ним виконано три дії по три рази з ротацією порядку (рисунок 3, таблиця 1).

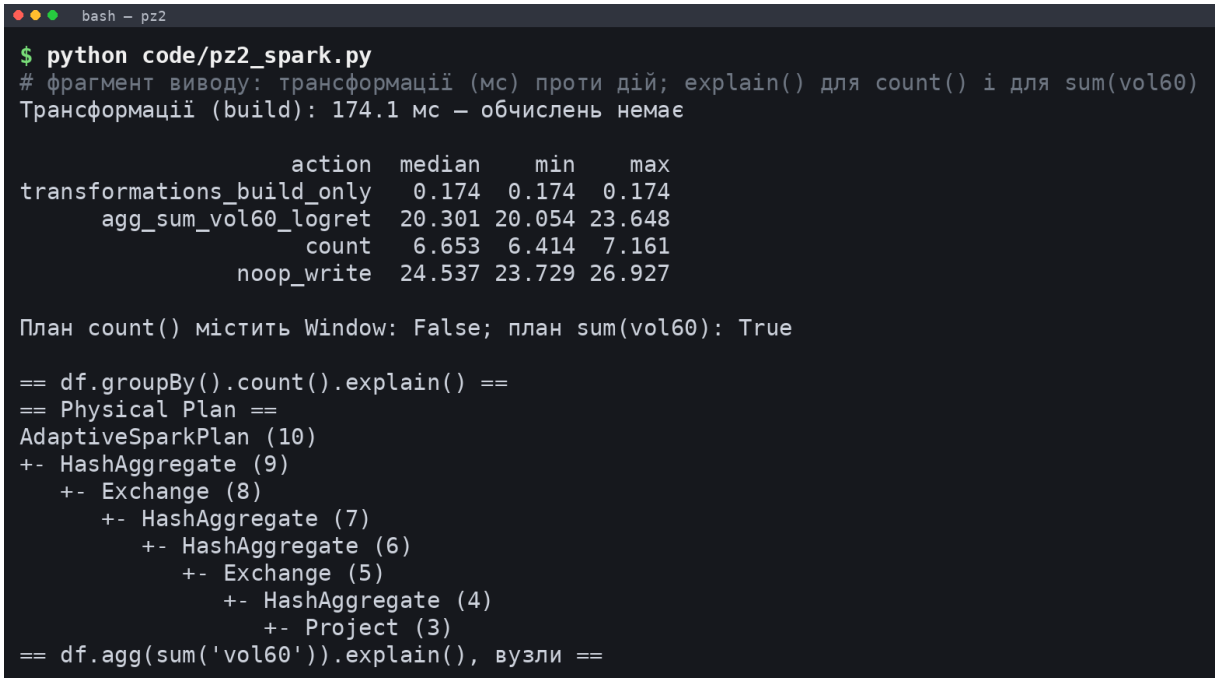


Рисунок 3 – Час трансформацій і дій та фізичний план count()

Таблиця 1 – Час дій над конвеєром і ефект кешування (медіана з трьох запусків)

Операція	Window у плані	Медіана, с	Мін.–макс., с
Лише трансформації `build()`	–	0,174	–
`count()`	ні	6,653	6,414–7,161
`agg(sum(vol60), sum(log_ret))`	так	20,301	20,054–23,648
`write.format("noop")`	так	24,537	23,729–26,927
Запит `heavy_query` без кешу	так	15,484	14,012–16,811
Матеріалізація кешу (`persist` + `count`)	так	28,950	21,209–29,722
Запит `heavy_query` над кешем	–	1,363	0,927–1,640

У плані `count()` немає вузлів `Window`: ColumnPruning відкидає непотрібні `log_ret` і `vol60` разом із вікнами. Повний прохід дає лише запис у `noop`, і різниця 6,653 і 24,537 с – це вартість двох вікон (обмін за

(year, month), сортування, `lag` і `stddev`). Граф повного проходу наведено на рисунку 4.

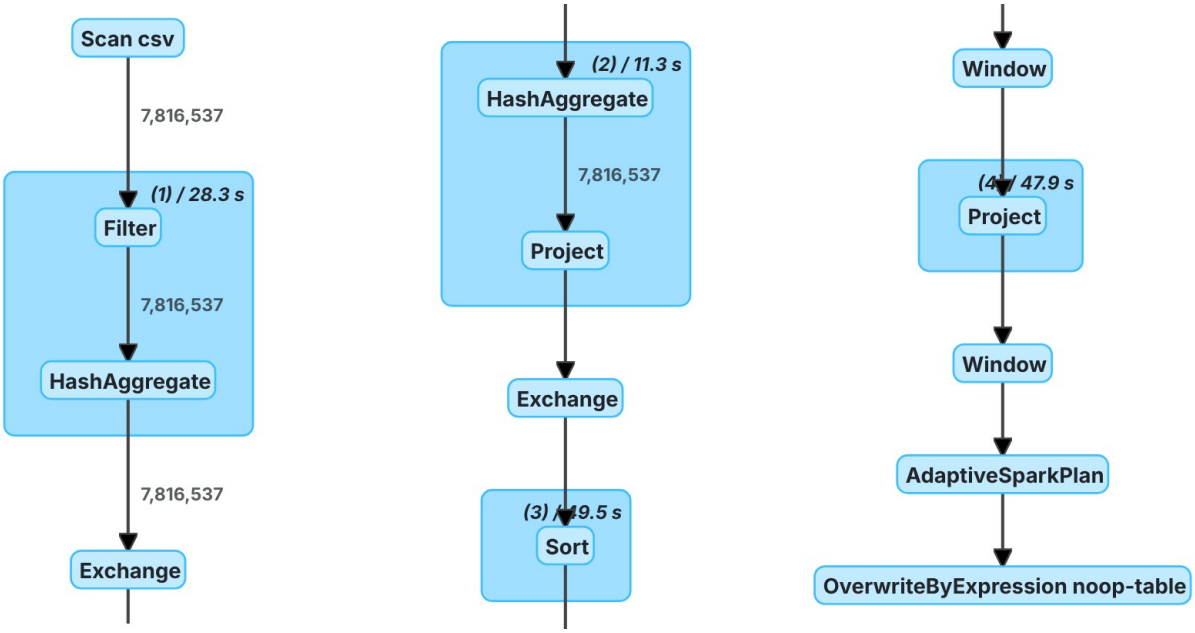


Рисунок 4 – Граф виконання повного проходу (noop-запис) у вкладці SQL Spark UI; граф розбито на три колонки для друку

Перший етап конвеєром виконує `Scan csv`, `Filter` і часткову дедуплікацію, другий після обміну за `Timestamp` її завершує, третій після обміну за (year, month) виконує `Sort` і два вузли `Window`.

Запит `heavy_query` (топ-10 діб за `rv` і обсяг за годиною) у трьох раундах з чергуванням порядку прискорився з кешем у 11,4 разу (15,484 проти 1,363 с); матеріалізація (28,950 с) окупається після 2,1 повтору, а над кешем виконано понад двадцять запитів. Кеш у вкладці Storage Spark UI займає 360,6 МіБ і всі 16 з 16 розділів (від 266 400 до 754 560 рядків, бо місяці розподіляються за хешем нерівномірно).

4 Розподілена аналітика

4.1 Групування і агрегація

Свічки згруповано за датою за правилами OHLC; перше й останнє значення взято `min_by`/`max_by` за міткою часу, бо після обміну порядок рядків у групі не гарантований:

```
d1 = df.groupBy("date").agg(
    F.min_by("Open", "Timestamp").alias("open"), F.max("High").alias("high"),
    F.min("Low").alias("low"), F.max_by("Close", "Timestamp").alias("close"),
    F.sum("Volume").alias("volume"),
    F.sqrt(F.sum(F.col("log_ret") ** 2)).alias("rv"))   # також n_minutes, n_flat
```

Отримано 5429 діб, 5427 з них повні (1440 хвилин); `rv` – добова реалізована волатильність. Погодинний агрегат має 130 276 рядків. Частка обсягу за годиною доби мінімальна о 03:00 UTC (2,85 %) і максимальна о 14:00 UTC (5,79 %), так само змінюється середній `rel_range` (0,000672 о 05:00, 0,001087 о 14:00).

4.2 Статистичні характеристики дохідностей

Асиметрію та надлишковий ексцес (`F.skewness`, `F.kurtosis`) на трьох масштабах наведено в таблиці 2.

Таблиця 2 – Асиметрія та надлишковий ексцес логарифмічних дохідностей

Ряд	n	Ст. відхилення	Асиметрія	Ексцес
1 хв, усі свічки	7 816 358	0,001733	3,92	7161,62
1 хв, без 2 найбільших |r|	7 816 356	0,001705	4,12	3260,81
1 хв, без 10 найбільших |r|	7 816 348	0,001665	−0,07	711,21
1 хв, лише з торгами	6 503 968	0,001900	3,58	5958,66
1 хв, з торгами, без 10 найбільших |r|	6 503 958	0,001825	−0,06	591,29
1 год	130 275	0,009522	−1,23	118,11
1 доба	5 428	0,040632	−1,31	25,46

Хвилинний ексцес 7161,62 створюють кілька хвилин. Дві найбільші за модулем, −0,617 і +0,617 (понад 350 σ), – миттєвий обвал 11.04.2013 о

10:57 UTC до 75,0 USD з поверненням до 138,99 USD при реальному обсязі 15,61 і 5,89 BTC. Додатна асиметрія 3,92 теж є ефектом викидів: стрибки вгору більші за модулем (+0,462 і +0,457 у січні 2012 року, +0,435 10.04.2013 проти −0,379, −0,370 і −0,252), і без десяти найбільших асиметрія стає −0,07. Добова асиметрія від'ємна через падіння 11.04.2013 (−0,664) і 12.03.2020 (−0,494). Динаміку волатильності показано на рисунку 5.

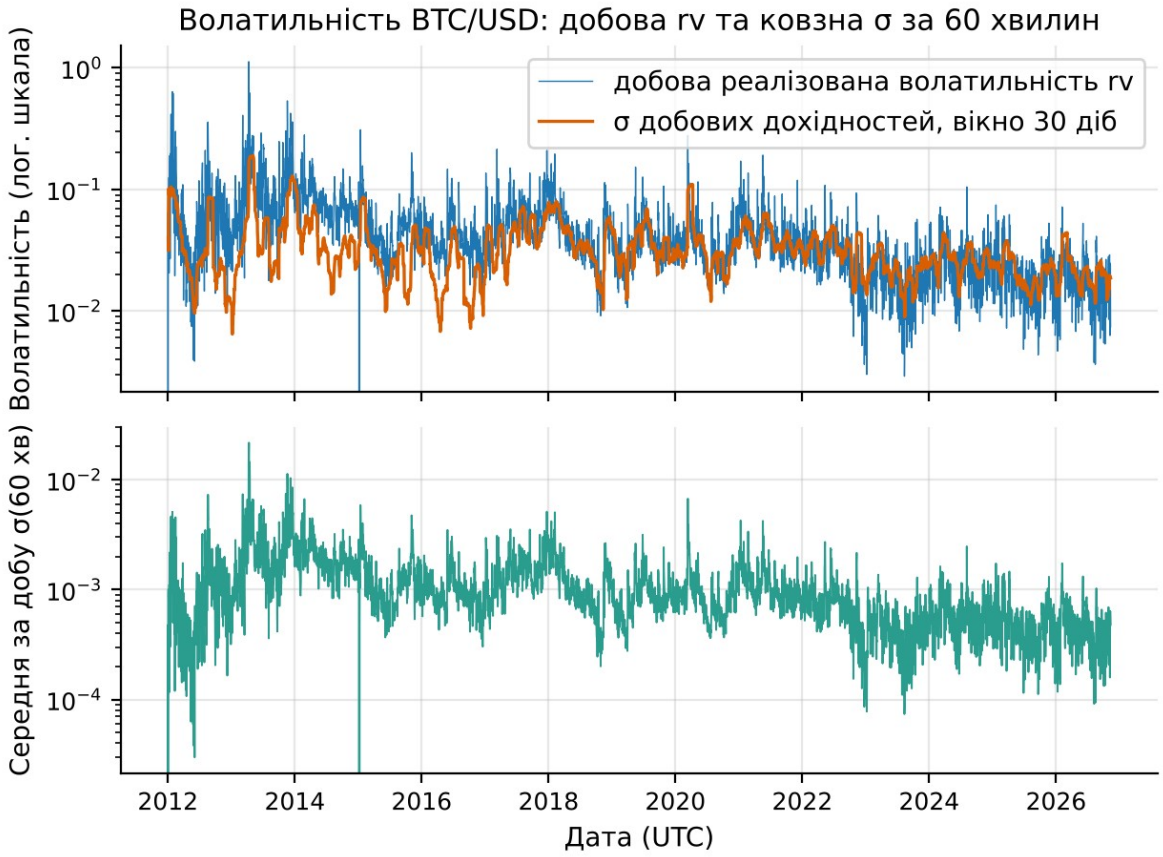


Рисунок 5 – Добова реалізована волатильність rv і середня за добу 60-хвилинна σ

Добова `rv` у 2013 році перевищувала 0,3 протягом 13 діб, а після 2023 року (1411 діб) має медіану 0,020, міжквартильний діапазон 0,015–0,027 і максимум 0,105 (05.08.2024). Сплески збігаються з подіями: 12.03.2020 `rv` 0,211 при 30-денній волатильності 0,094, 19.05.2021 – 0,191 при 0,053. Провали майже до нуля у 2012 році і 6–8.01.2015 відповідають добам майже без угод, а не затишшю.

4.3 Фільтрація і сортування: найволатильніші доби

Повні доби відсортовано за спаданням `rv` і `hl_rel` з фільтром `flat_share < 0,5` і без нього. Без фільтра доби 27.01.2012 і 31.01.2012 посідають четверте і шосте місця за `rv` (0,6358 і 0,6160), хоча 98,1 % і 97,2 % їх хвилин без угод: поодинокі угоди переносять увесь накопичений рух ціни, тож `rv` тут відображає неліквідність. Шість із десяти діб за `rv` припадають на квітень 2013 року (лідер 11.04.2013, `rv` 1,1260), коли ціна впала з 259,34 до 45 USD. Єдина доба після 2013 року в рейтингу за розмахом – 13.03.2020 (44,21 %).

5 Spark SQL

5.1 Тимчасові представлення та запити

Над представленнями `btc` (кешований хвилинний DataFrame) і `btc_daily` (добовий агрегат) виконано сім запитів: Q1 – річна статистика, Q2 – топ-10 діб за розмахом, Q3 – обсяг за днем тижня і годиною, Q4 – місячна сезонність, Q5 – доба з максимальним обсягом у році (`ROW_NUMBER()`), Q6 – шість кризових діб, Q7 – усі доби зі сплеском обсягу з 2018 року. Кожен запит тривав 0,633–1,203 с. Фрагмент Q7:

```
SELECT * FROM (
  SELECT date, ROUND(100*ret_1d,2) AS ret_pct,
    ROUND(volume / AVG(volume) OVER (ORDER BY date ROWS
      BETWEEN 30 PRECEDING AND 1 PRECEDING), 2) AS volume_x_prev30_mean
  FROM btc_daily)
WHERE date >= DATE'2018-01-01' AND volume_x_prev30_mean >= 4.63
```

Вікно рахується в підзапиті над усіма добами, а відбір – зовні, інакше середнє бачило б лише відібрані дати. План Q1 читає дані через `InMemoryTableScan`, тож CSV і вікна повторно не обчислюються.

5.2 Результати запитів

Результат Q1 наведено в таблиці 3.

Таблиця 3 – Річна статистика хвилинних свічок (запит Q1)

Рік	Хвилин	Середня ціна, USD	Максимум High, USD	Мінімум Low, USD	Обсяг, BTC	Flat, %
2012	527 039	8,23	16,41	3,80	567 949	94,95
2013	525 600	187,43	1 163,00	12,77	5 031 143	39,16
2014	525 600	525,57	995,00	275,00	5 024 184	24,24
2015	525 600	272,25	502,00	152,40	5 525 344	29,08
2016	527 040	565,83	980,74	352,00	1 992 258	33,19
2017	525 600	3 948,33	19 666,00	751,34	4 698 753	8,04
2018	525 600	7 524,76	17 234,99	3 122,28	3 932 327	3,77
2019	525 600	7 352,19	13 880,00	3 322,19	2 994 600	3,28
2020	527 040	11 074,63	29 300,00	3 850,00	3 074 321	1,55
2021	525 600	47 386,18	69 000,00	27 734,00	1 802 998	0,70
2022	525 600	28 228,66	48 234,00	15 479,00	833 815	2,66
2023	525 600	28 811,76	44 729,00	16 496,00	681 687	3,95
2024	527 040	65 895,12	108 364,00	38 505,00	807 910	3,33
2025	525 600	101 639,98	126 272,00	74 434,00	639 946	1,13
2026	452 378	74 103,97	97 939,00	57 734,63	636 371	0,34

Обсяг у BTC упав з 5 525 344 (2015) до 639 946 (2025) через зростання ціни. За Q3 максимум хвилинного обсягу з 2018 року припадає на будні о 14:00 UTC (6,1661 BTC у середу), мінімум – на неділю о 02:00 UTC (1,532 BTC). За Q4 найбільша середня добова `rv` у січні (0,03977) і березні (0,03912), найменша – у жовтні (0,02342). За Q5 рекордна доба за обсягом – 18.12.2013 (137 070 BTC).

За Q6 у дні обвалу 12–13.03.2020, падіння 19.05.2021, заморожування виведення коштів Celsius 13.06.2022 і колапсу FTX 08–09.11.2022 обсяг перевищував середній за 30 попередніх діб у 4,63–9,27 разу (12.03.2020: −49,40 % при 9,27). Чи такий сплеск властивий лише кризам, перевірено запитом Q7 з порогом 4,63 (рисунок 6).

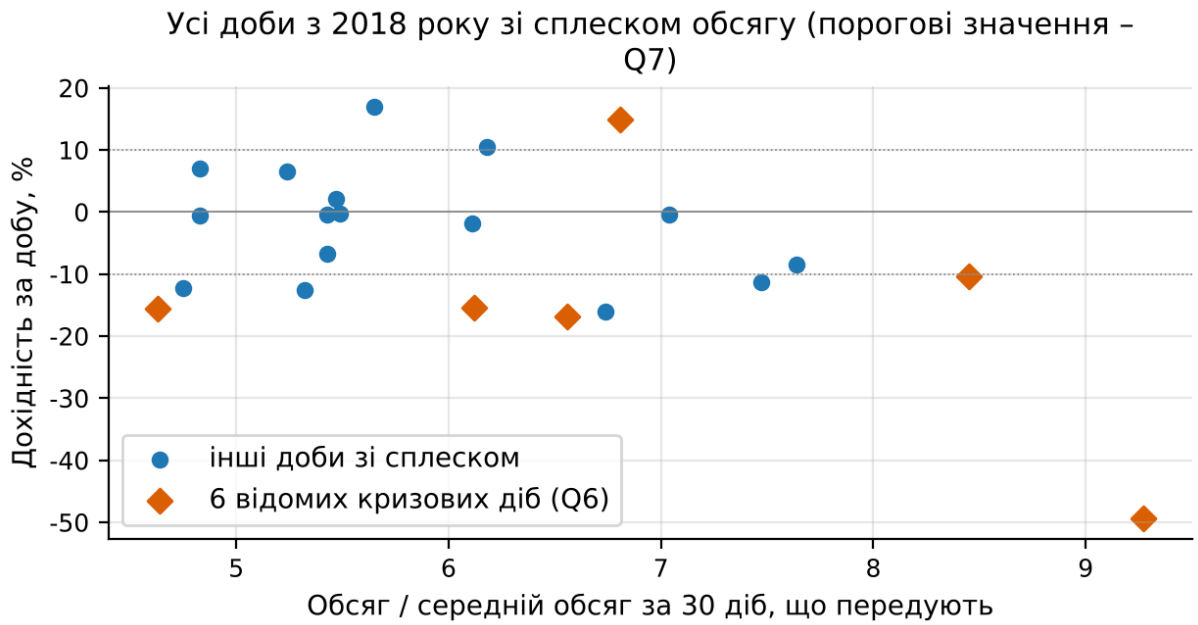


Рисунок 6 – Доби з 2018 року з обсягом не менше 4,63 середнього за 30 діб, що передують (запит Q7)

Поріг перевищили 22 доби з 3237 (0,68 %), медіана відношення 5,88. З них 12 мають |дохідність| ≥ 10 %, а 6 – менше 5 %, наприклад 12.05.2022 (−0,36 %) і 15.07.2025 (−1,78 %): кризи супроводжуються сплеском обсягу, але майже половина сплесків не має великої зміни ціни.

6 Порівняння Hadoop MapReduce та Spark

6.1 Наскрізний бенчмарк Hadoop Streaming, Hive, pandas і Spark

Бенчмарк `bench_all.py` на тій самій машині виконував дві задачі: T1 – кількість хвилин і обсяг за роками, T2 – добові OHLC. Порівнювалися Hadoop Streaming 3.3.6 (YARN і HDFS на одному вузлі, mapper і reducer з ПЗ1), Hive 3.1.3 на рушії MR (`bench_yearly.hql`, `bench_daily.hql`), pandas і PySpark (драйвер 3 ГБ, 8 shuffle-розділів) над тим самим CSV. Вимірювався час до записаного результату і окремо старт рушія («обробка» – різниця медіан); п'ять кіл після прогрівального виконано з циклічним зсувом порядку рушіїв, усі 48 запусків пройшли звірку з pandas (таблиця 4, рисунок 7).

Таблиця 4 – Наскрізний час виконання задач (медіана з п'яти запусків), с

Задача	Рушій	Загальний час	Мін.–макс.	Старт	Обробка
T1	Hadoop Streaming	27,20	25,54–28,18	15,47	11,73
T1	Hive (MR)	35,19	34,23–36,68	11,88	23,31
T1	pandas	8,46	8,14–9,64	0,43	8,03
T1	PySpark	12,88	12,57–14,29	6,29	6,58
T2	Hadoop Streaming	24,08	22,42–25,47	15,47	8,62
T2	Hive (MR)	43,38	42,95–45,18	11,88	31,49
T2	pandas	9,21	8,50–10,18	0,43	8,78
T2	PySpark	14,04	13,58–14,86	6,29	7,75

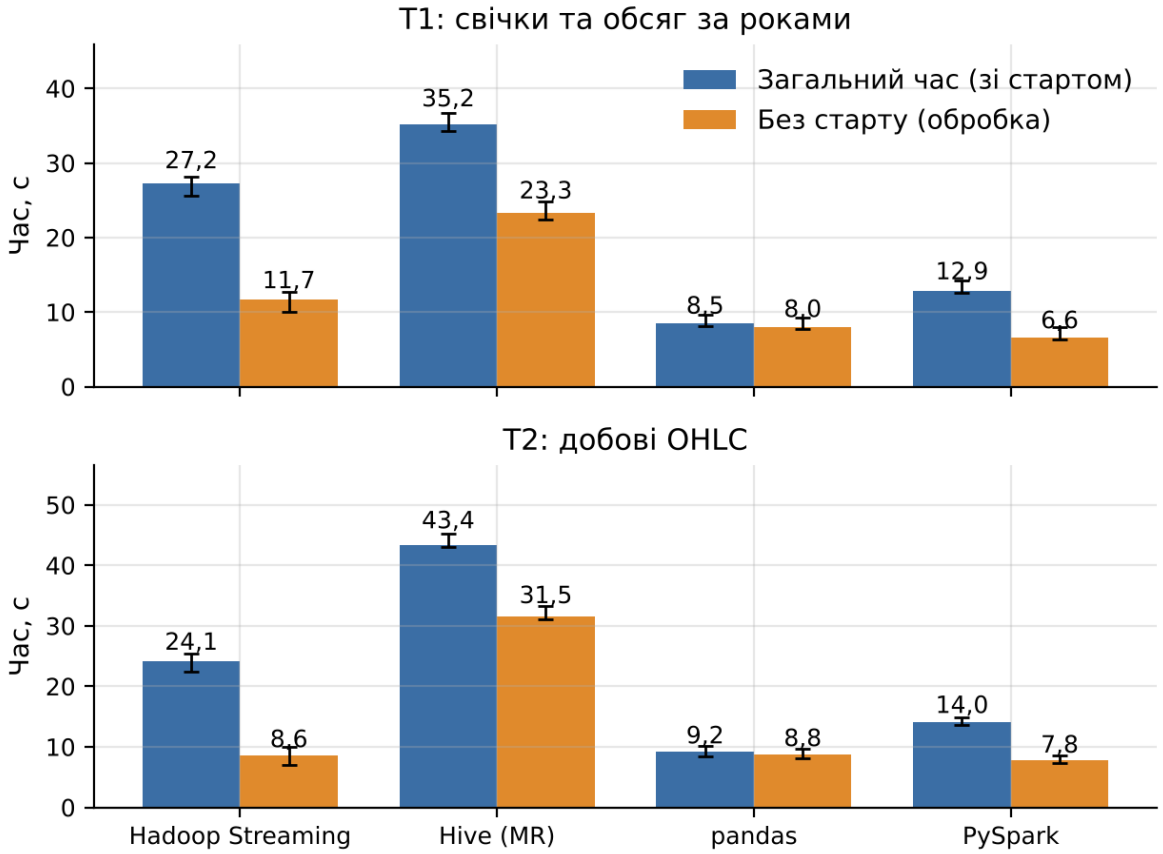


Рисунок 7 – Наскрізний час задач T1 і T2 у чотирьох рушіях: загальний і без старту

Spark виконав задачі у 1,7–2,1 раза швидше за Hadoop Streaming, і більшу частину різниці дає старт: 15,47 с на подання завдання в YARN проти 6,29 с на створення SparkSession. Без старту Spark швидший на T1 (6,58 проти 11,73 с) і майже рівний на T2 (7,75 проти 8,62 с). За загальним часом виграє pandas (старт 0,43 с), але без старту Spark на чотирьох ядрах швидший (6,58 проти 8,03 с і 7,75 проти 8,78 с). Обмеження: один вузол, запис у HDFS для Hadoop і Hive проти локального диска, п'ять вимірювань.

6.2 Вимірювання в межах сесії Spark

У запущеній сесії виміряно Spark і pandas (порядок S, P, S, P, S, P), ті самі задачі над кешем і пікову RSS (кожні 50 мс) – таблиця 5.

Таблиця 5 – Час і пікова пам'ять Spark і pandas у межах однієї сесії (медіана з трьох запусків)

Задача	Рушій	Медіана, с	Мін.–макс., с	Пікова пам'ять, МБ
T1	pandas	3,556	3,524–3,707	783,1
T1	Spark, читання CSV	3,588	3,023–3,944	1483,3 (JVM) + 124,7 (Python)
T1	Spark, кеш	0,549	0,428–0,729	–
T2	pandas	4,884	4,851–5,449	879,0
T2	Spark, читання CSV	4,547	4,220–4,841	1496,7 (JVM) + 128,9 (Python)
T2	Spark, кеш	0,618	0,563–0,986	–

У сесії Spark і pandas рівні на T1 (3,588 проти 3,556 с), на T2 Spark трохи швидший; результати збігаються (обсяг з відносною похибкою до 10^{-6}). Над кешем задачі тривають 0,549 і 0,618 с, отже без кешу основний час займає читання CSV. Серії таблиць 4 і 5 не об'єднано: у бенчмарку кожен запуск – новий процес з холодною JVM і записом у файл. Пікова RSS JVM (1483,3–1496,7 МБ) вища, ніж у pandas, але не є мінімальною потребою: з драйвером 2 ГБ ті самі задачі вкладаються в 923,3 і 1026,1 МБ, бо JVM звільняє пам'ять ліниво; pandas при читанні частинами по 500 000 рядків виконує T1 з піком 197,0 МБ.

6.3 Порівняння за критеріями методичних вказівок

Порівняння за п'ятьма критеріями зведено в таблицю 6.

Таблиця 6 – Порівняння Hadoop MapReduce і Apache Spark

Критерій	Hadoop MapReduce	Apache Spark
Модель обчислень	Жорстка послідовність map – shuffle – reduce на кожне завдання; багатокроковий аналіз – ланцюжок завдань	DAG з довільної кількості трансформацій; вузькі операції об'єднуються в етап (рисунок 4: Scan, Filter і часткова агрегація в одному етапі)
Використання оперативної пам'яті	Потокова обробка записів, проміжні дані на диску; пам'ять обмежена розміром контейнерів YARN	Проміжні дані й кеш у пам'яті; JVM 1483,3–1496,7 МБ при драйвері 8 ГБ, 923,3–1026,1 МБ при 2 ГБ; кеш 360,6 МіБ
Продуктивність (таблиця 4)	27,20 і 24,08 с, з них 15,47 с старт	12,88 і 14,04 с, з них 6,29 с старт
Зручність програмування	Окремі програми mapper і reducer, ручний розбір рядків і вибір першого та останнього значення для OHLC	Один вираз `groupBy().agg()` з `min_by`/`max_by` або SQL-запит; DataFrame API і SQL дають однаковий план
Повторне використання даних	Кожне завдання заново читає вхідні дані з HDFS; кешу між завданнями немає	`persist()`: запит у 11,4 разу швидший (15,484 → 1,363 с), окупність 2,1 повтору

7 Візуалізація

Скрипт `make_figures.py` будує графіки з агрегатів, переданих у Python (5429 добових рядків замість 7 816 537); у звіті наведено три з них – рисунки 5, 6 і 7.

Висновки

1. Розгорнуто Apache Spark 4.3.0 (`local[*]`, 4 ядра) з PySpark, Java 21 і Jupyter Notebook.
2. Датасет завантажено у DataFrame: 7 816 537 записів, 6 атрибутів, 0 пропусків; виведення типів подовжує розбір у 2,14 разу.
3. Попередня обробка: 0 дублікатів, явні типи, похідні ознаки у вікнах за (year, month) з крайовим ефектом на 0,14 % рядків, flat-хвилини (16,79 %) позначено. Ліниві обчислення підтверджено планами: `count()` вікон не обчислює (6,653 с проти 24,537 с повного проходу).

4. Реалізовано фільтрацію, групування, сортування, агрегацію та моменти: хвилинні ексцес 7161,62 і асиметрія 3,92 створені кількома викидами (без десяти – 711,21 і −0,07).
5. Виконано сім SQL-запитів: у кризових добах обсяг перевищував середній за 30 діб у 4,63–9,27 разу, але з 22 діб зі сплеском з 2018 року лише 12 мали зміну ціни від 10 %.
6. Spark виконує трансформації конвеєром в етапах DAG без запису проміжних даних на диск, займає більше пам'яті (1483,3–1496,7 МБ проти 783,1–879,0 МБ у pandas), наскрізно швидший за Hadoop Streaming (12,88 і 14,04 проти 27,20 і 24,08 с) переважно за рахунок старту, задає OHLC одним виразом, а кеш прискорив запит у 11,4 разу.
7. На одній машині з файлом, що вміщується в пам'ять, pandas найшвидший за загальним часом, а перевага Spark проявляється в паралельній обробці та повторних запитах над кешем.